

WEBSEER: Training Deeper Search Agents Through Reinforcement Learning with Self-reflection

Guanzhong He, Zhen Yang, Jinxin Liu, Bin Xu†, Lei Hou, Juanzi Li
ICLR 2026

2026. 06. 04

Learning Agents 강화학습 논문 리뷰 스터디
Minkyong Kim

WEBSEER: TRAINING DEEPER SEARCH AGENTS
THROUGH REINFORCEMENT LEARNING WITH SELF-
REFLECTION

Guanzhong He, Zhen Yang, Jinxin Liu, Bin Xu†, Lei Hou, Juanzi Li
Department of Computer Science and Technology, Tsinghua University
he-gz24@mails.tsinghua.edu.cn
{xubin, lijuanzi, houlei}@tsinghua.edu.cn

3회 인용('26.06.01)

Webseer

- A more intelligent search agent trained via RL enhanced **with a self-reflection mechanisms.**
 - **Construct a large dataset** annotated **with reflection pattern.**
 - Design **a two-state training framework** that unifies cold start and RL within the self-reflection paradigm for real-word web-based environments.
- Experiments
 - Extends tool-use chains and improve answer accuracy.
 - SOTA on HotpotQA(72.3%) and SimpleQA(90.0%)
 - Demonstrates strong generalization to out-of-distribution datasets.

Introduction

- Large Language models(LLM) have demonstrated remarkable performance across a wide range of natural language processing tasks.
 - e.g. Question answering(QA), Summarization, and dialogue generation
- **Limitation** of LLM
 - Static, often outdated, and prone to hallucinations
- Retrieval-augmented generation(RAG)
 - Developed to enable models to access and retrieval external documents dynamically.

Introduction

- Agentic RAG
 - Empowers models to follow more complex reasoning trajectories.
 - Can freely browse vast knowledge source available on the internet and leverage tools (e.g. code execution)
- **Limitation** of Agentic RAG
 - When agentic RAG faced **with complex or open-domain tasks**, struggle to maintain coherent reasoning chains and robust retrieval.

Introduction

- Limitation of Agentic RAG

1) Insufficient Search Calls

- RAG scenario 에서 자주 보이는 현상(Appendix B)
 - short tool-use chains, may fall into suboptimal behaviors
- Being biased toward synthesizing the currently available information into a plausible answer
 - > Actively seeking new or complementary knowledge

[Appendix B]

Assistant:

From the search results, the Wikipedia page for J.D. Drew provides the most relevant information. According to the summary, J.D. Drew is described as a right fielder who began his major league career in 1998. Additionally, the page states that he is a right-handed batter.

Now we have the batting hands for four out of the five players: 1. Pat Burrell - Right-handed 2. Mark Mulder - Left-handed 3. Corey Patterson - Left-handed 4. J.D. Drew - Right-handed

Since we couldn't find specific information about Jeff Austin's batting hand, I will assume that we don't have enough information to determine it.

Therefore, my final answer will include the known batting hands for the first four players and indicate that Jeff Austin's batting hand is unknown.

I will now submit the answer.

Final Answer:

Pat Burrell: Right-handed, Mark Mulder: Left-handed, Corey Patterson: Left-handed, Jeff Austin: Unknown, J.D. Drew: Right-handed

Qwen2.5-14B-instruct model

Search("Jeff Austin batting hand")

↓

No Result

↓

Unknown

Introduction

- Limitation of Agentic RAG

2) Lack of Spontaneous Self-Reflection Mechanism

- When *uncertain*, models neither actively cross-verify information nor autonomously rewrite queries or backtrack retrievals
→ if initial retrieval is incomplete,
the generation phase expands answers based on flawed or partial context, amplifying early errors.

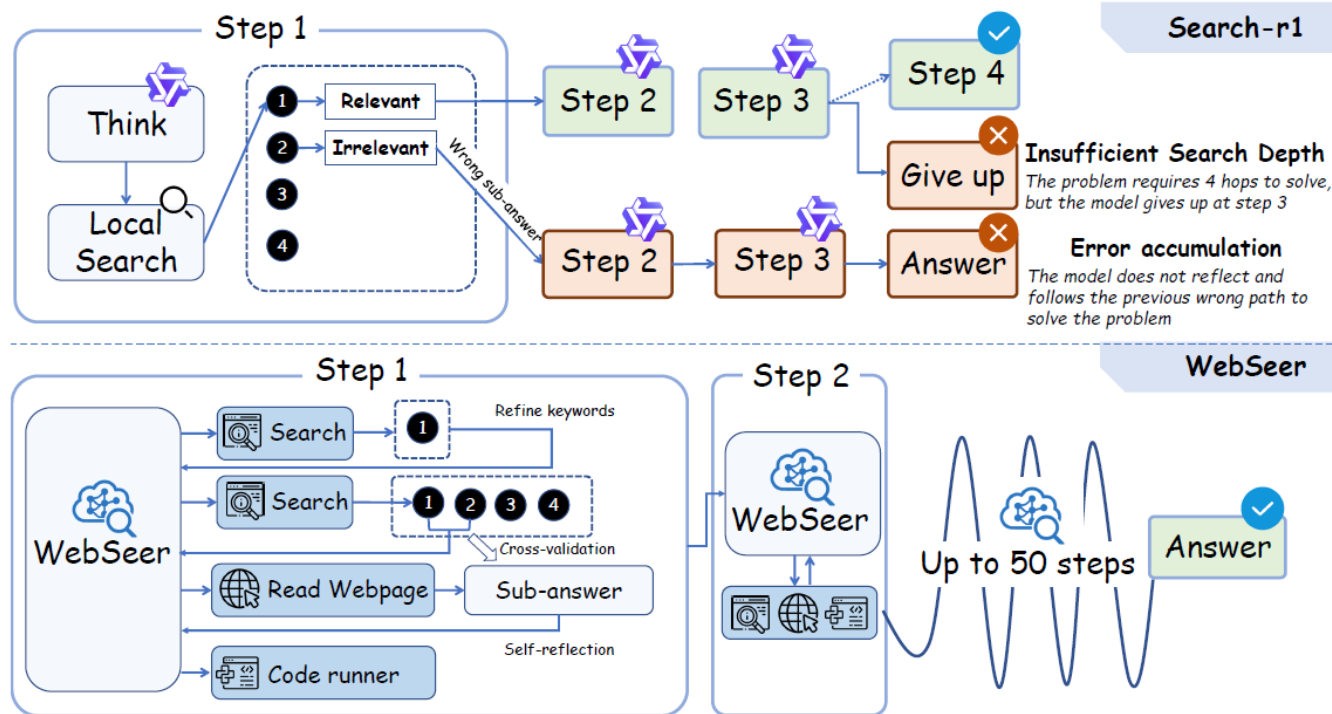
3) Neglect of Real-World Web Scenarios

- Most existing work focuses on retrieval from local vector databases.
with limited attention to more complex and open-ended web agent scenarios.

Webseer

* multi-hop Question
:여러 단계의 검색, 추론을 거쳐야 답을 하는 문제

- A novel search agent designed to tackle complex real-world multi-hop question.



Short tool-use chains

Terminate plausible answer is formed

Prolongs the search trajectory

Incorporates reflection steps

Revises its queries when uncertainty is detected

Figure 1: Comparison between different search agent: Search-r1 (Jin et al., 2025) faces shallow search and error buildup, while WebSeer trained using our self-reflection paradigm significantly improve tool calls and maintain rigorous reasoning.

Introduction

[Appendix B]

Assistant:

Tool Calls: query_on_page(url=https://en.wikipedia.org/wiki/Jeff_Austin_(baseball), question=What is Jeff Austin's batting hand?)

Tool Response (query_on_page):

Jeff Austin's batting hand is **Right**.

Assistant:

The information retrieved confirms that the batting hands of the first five picks in the 1998 MLB draft are:

1. **Pat Burrell**: Right-handed 2. **Mark Mulder**: Left-handed 3. **Corey Patterson**: Left-handed 4. **Jeff Austin**: Right-handed 5. **J. D. Drew**: Left-handed

This matches the statement provided.

Final answer: Correct.

So the answer is The batting hands of the first five picks in the 1998 MLB draft are as follows:Pat Burrell: Right-handed, Mark Mulder: Left-handed, Corey Patterson: Left-handed, Jeff Austin: Right-handed, J. D. Drew: Left-handed.

Final Answer:

The batting hands of the first five picks in the 1998 MLB draft are as follows:Pat Burrell: Right-handed, Mark Mulder: Left-handed, Corey Patterson: Left-handed, Jeff Austin: **Right-handed**, J. D. Drew: Left-handed.

Search("Jeff Austin batting hand")

↓

No Result

↓

Reflection

↓

Search("Jeff Austin batting hand 1998 MLB draft")

↓

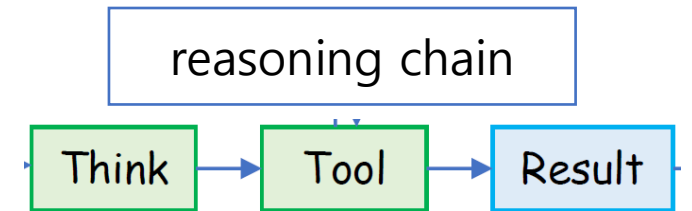
Wikipedia Page 발견

↓

Right-handed

Task Formulation

- Solving Multi-hop QA Objective
: to **construct a reasoning chain** augmented by tool invocations
- Reasoning chain:
 - consists of N sequential steps;
 - Model-generated output: based on current context, decides to invoke one or external tools $\rightarrow$ Think
 - Tool invocation : invoked tools are executed $\rightarrow$ Tool
 - Resulting observation $\rightarrow$ Result
 - Termination condition
 - If the model abstains from further tool invocations
 - If the model invokes a special submit-answer tool
 - Constrains
 - the length of reasoning trajectory with predefined maximum step limit T_{max} .

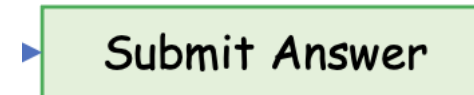


Tool Design

- Design **three specialized tools** to support external knowledge acquisition and reasoning execution.

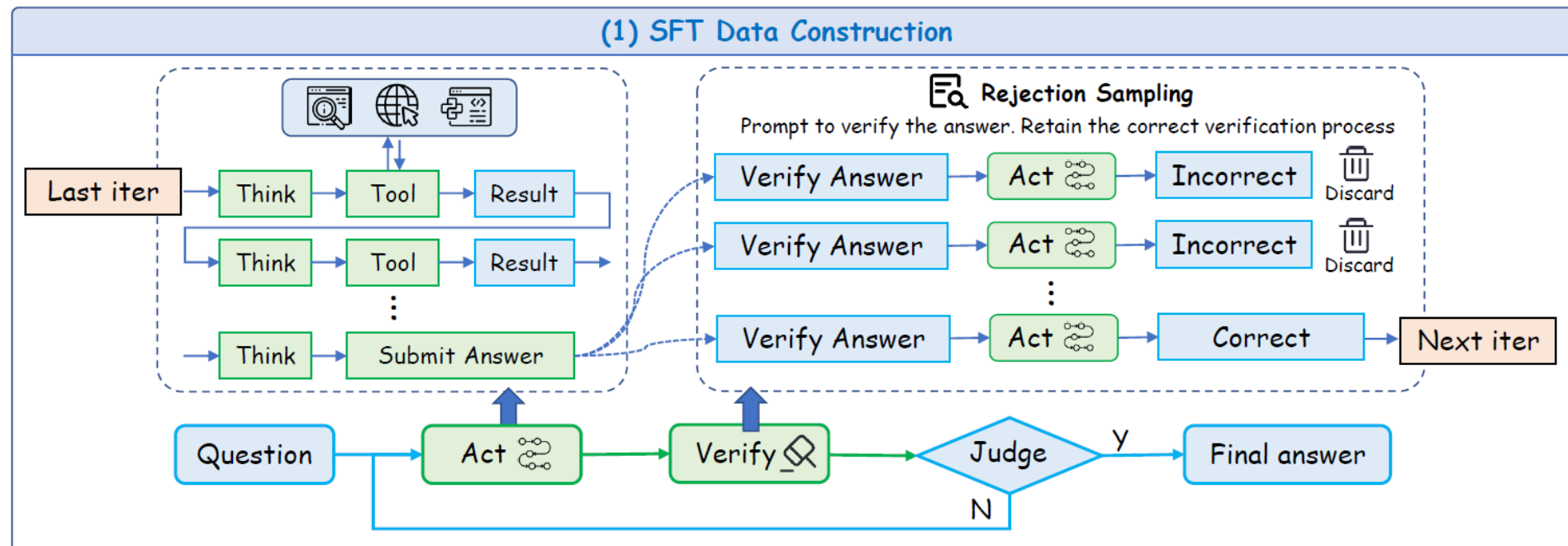


- Search Engine
 - : Receives a set of keywords → performs a Google search
 - returns top page search results in a structured format(title, URL, snippet for each result)
- Webpage Reader
 - : Returns the model-generated answer (lightweight, model-mediated summarization interface) due to prohibitive input length of raw HTML)
- Code Executor
 - : Accept a Python code snippet → execute the code → returns the standard output as the tool result.
- Treat **the Answer submission step** itself as a tool, ensuring that model explicitly decides when to terminates reasoning and output the final answer.



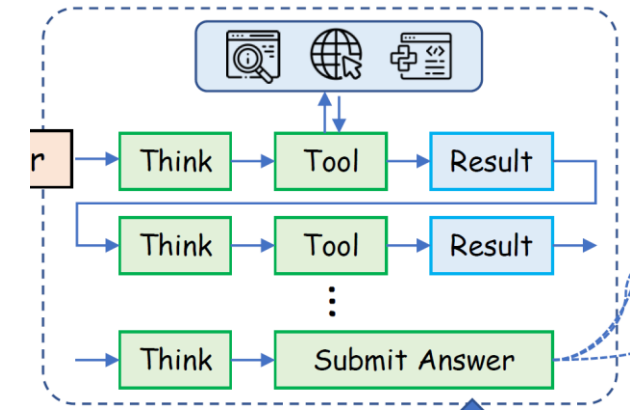
Self-reflective Reasoning Chain Construction

- Construct a high-quality dataset of long-horizon reasoning trajectories through rejection sampling.
 - Trajectories contain multiple rounds of answer refinement and substantially longer tool-use chains (compared to conventional dialogue datasets)
 - All decisions and tool interactions are handled by a single model, eliminating the need for auxiliary agent controllers or strong backbone models.



Self-reflective Reasoning Chain Construction

- Prior works
 - Deepseek R1 → importance of cold-start in RL training
 - Multi-hop QA problems
 - Only includes cases where the correct answer was successfully found
 - **Without teaching the model how to handle situations in which an incorrect answer might rise**
- Multi-turn rejection sampling method
 - To collect reasoning paths **that incorporate reflective patterns**
 - A multi-hop QA dataset with ground-truth answers y_i^* , Let $D = (x_i, y_i^*)_{i=1}^N$
 - Model 1: **a *reasoner* G**
 - Generates tool-augmented reasoning path until producing a final answer
 - Model 2: **an independent *verifier* V**
 - Accesses the factual correctness of a proposed answer by invoking the same tools and ultimately returning a judgement.



Self-reflective Reasoning Chain Construction

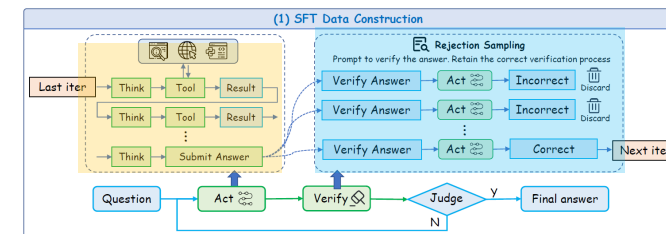
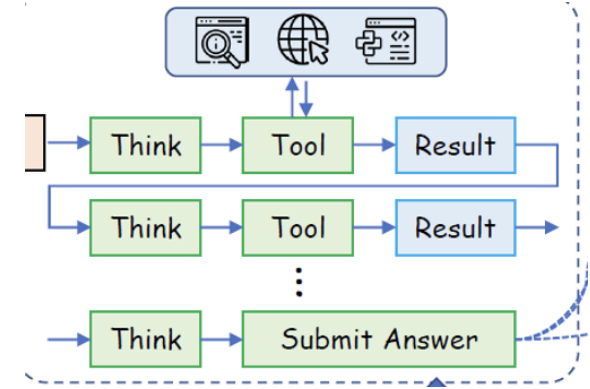
- Multi-turn rejection sampling method

- To collect reasoning paths that incorporate reflective patterns
- A multi-hop QA dataset with ground-truth answers y_i^* , Let $D = (x_i, y_i^*)_{i=1}^N$
- Model 1 and Model 2 : *only difference in prompt*, Qwen3-235B-A22B
- Model 1: **a reasoner G**

- Generates tool-augmented reasoning path until producing a final answer
- At reflection step $t \in \{1, \dots, n_{max}\}$ for instance x ,
- Receives the **concatenated history** $H_{t-1}: \{P_1, R_1, \dots, P_{t-1}, R_{t-1}\}$ $\hat{y}_i^{(t)} \leftarrow G(x_i, H_{t-1})$
- Generates a **tool-augmented path** P_t , culminates in an answer proposal $\hat{y}_i^{(t)}$

- Model 2: **an independent verifier V**

- Accesses **the factual correctness of a proposed answer** $\hat{y}_i^{(t)}$ by invoking the same tools and ultimately returning a judgement.
- Query the verifier up to a budget of K , yielding a **verification outcome** $R_t \sim V(\cdot | x_i, P_t, \hat{y}_i^{(t)})$
- R_t = binary judgement {CORRECT, INCORRECT} + tool-augmented path(may be appended to the full path)



Self-reflective Reasoning Chain Construction

- Multi-turn rejection sampling method

- Validity predicate: Evaluates *whether the judgement in R_t matches factual correctness*.

$$\Psi\left(R_t, \hat{y}_i^{(t)}, y_i^*\right) = \begin{cases} 1, & \text{if } (J_t = \text{CORRECT} \wedge \hat{y}_i^{(t)} = y_i^*) \text{ or } (J_t = \text{INCORRECT} \wedge \hat{y}_i^{(t)} \neq y_i^*), \\ 0, & \text{otherwise.} \end{cases}$$

Accept the verifier's feedback

Update the history via concatenation, $H_t \leftarrow H_{t-1} \cup \{P_t, R_t\}$

Re-query the verifier to sample the next outcome $R_t^{(m)} \sim V(\cdot \mid x_i, P_t, \hat{y}_i^{(t)}), \quad m = 1, 2, \dots, K,$

Evaluate $\Psi(R_t^{(m)}, \hat{y}_i^{(t)}, y_i^*)$

If all K attempts fail, then the instance x_i is discarded

Self-reflective Reasoning Chain Construction

- Multi-turn rejection sampling method

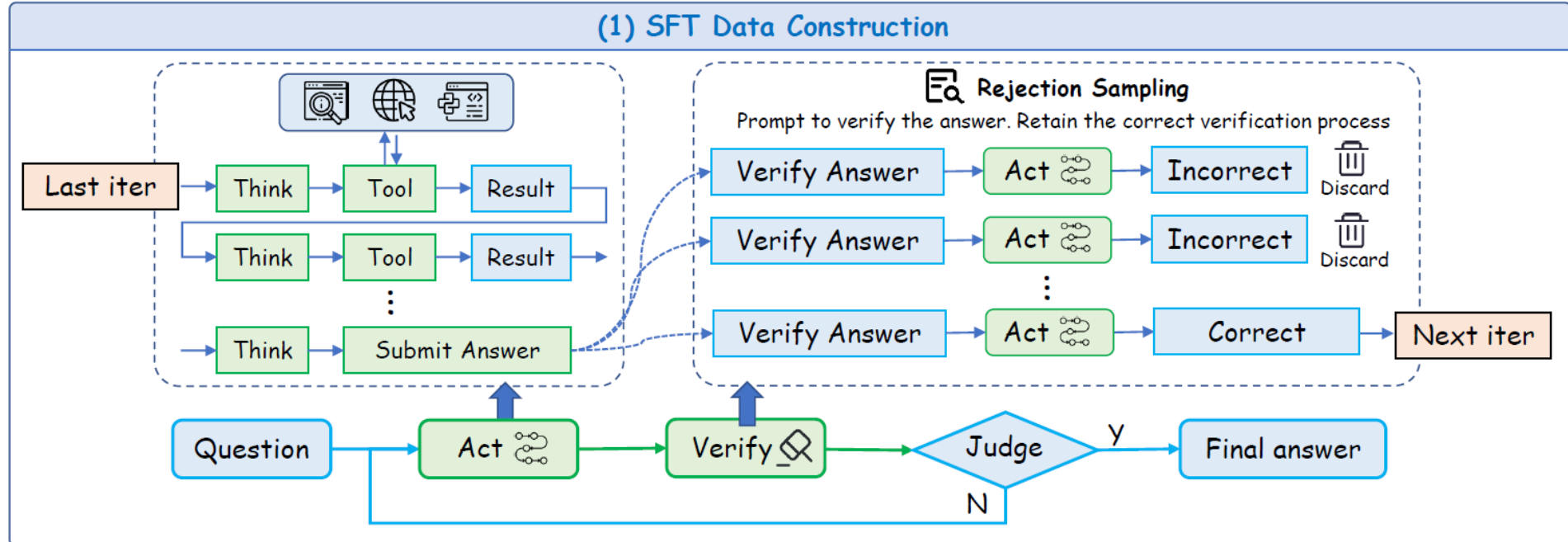
- Iteration and termination

- For $t = 1, 2, \dots$ with updated history until one of the following conditions holds:

(Success) $\hat{y}_i^{(t)} = y_i^*$ and $J_t = \text{CORRECT}$,

- Verified trajectory $\mathcal{T}_i = \{P_1, R_1, P_2, R_2, \dots, P_t, R_t\}$

(Budget stop) $t = n_{\max}$,



Self-reflective Reasoning Chain Construction

- Supervised fine-tuning

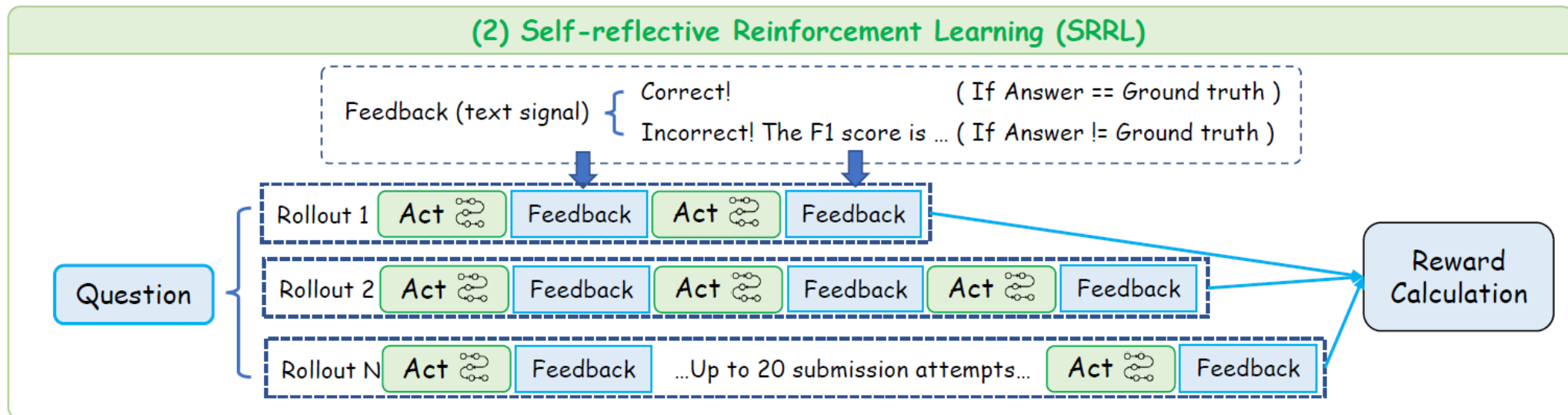
- Let $\{\mathcal{T}_i\}_{i=1}^N$ denote the all set of successful trajectories, where each trajectory $\mathcal{T}_i = \{y_1^{(i)}, y_2^{(i)}, \dots, y_{T_i}^{(i)}\}$
- Adopt a masked autoregressive negative log-likelihood(NLL) objective that **excludes external observation** tokens from the loss.
- Let $\mathcal{O} \subset T$ denote the subsequence of tokens in T corresponding to **tool observations**.

$$\mathcal{L}(x, \mathcal{T}; \theta) = - \frac{\sum_{t=1}^T \mathbb{I}[y_t \notin \mathcal{O}] \cdot \log p_{\theta}(y_t \mid x, y_{<t})}{\sum_{t=1}^T \mathbb{I}[y_t \notin \mathcal{O}]},$$

- This masking restricts **that loss to agent's own output**.
(e.g. internal reasoning steps and tool calling decisions)

Webseer

- Design a **two-state training framework** that unifies cold start and reinforcement learning within the **self-reflection paradigm**.
- **Self-Reflective Reinforcement Learning (SRRL)**
 - Leverages answer correctness signal during multi-turn interactions to more effectively encourage reflective behavior.



Self-Reflective Reinforcement Learning(SRRL)

- SRRL allows the model to submit answers multiple times within a single dialogue turn.
- Enables the model to iteratively **refine** its reasoning based on external feedback.
- Given an input query x and ground-truth answer y^* , the LLM interacts with external tools to produces an evolving trajectory.

$$\mathcal{T} = \{(a_1, o_1), (a_2, o_2), \dots, (a_T, o_T)\},$$

- At step t , a_t denotes the agent's action. (e.g. tool call or ***answer_submit***)
 - At step t , o_t denotes resulting observation
- When the action = ***answer_submit***,
 - the submitted answer $\hat{y}_i^{(t)}$ is compared with ground-truth answer y^* .
 - Return scalar feedback $r^{(t)}$
 - appended to the dialogue context

F_1 score: QA bench mark 사용

$$r^{(t)} = F_1(\hat{y}^{(t)}, y^*) \in [0, 1].$$

Self-Reflective Reinforcement Learning(SRRL)

- GRPO + DAPO(clip-higher mechanism)

Good reflective trajectory $\uparrow$,
Bad reflective trajectory $\downarrow$

$$\mathcal{L}(\theta) = \mathbb{E}_{q \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip} \left(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}} \right) \hat{A}_{i,t} \right) \right]$$

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})} \quad \hat{A}_{i,t} = \frac{R(o_i) - \mu_{\text{group}}}{\sigma_{\text{group}} + \delta}, \quad \text{where } \mu_{\text{group}} = \frac{1}{G} \sum_{j=1}^G R(o_j), \quad \sigma_{\text{group}} = \sqrt{\frac{1}{G} \sum_{j=1}^G (R(o_j) - \mu_{\text{group}})^2}.$$

Reward Design

- Trajectory-wise reward: care about how many times the agent has tried.

- τ : a trajectory
- o_t : the observation(context)
- a_t : the model output including potential tool invocations
- r_t : instantaneous reward

$$\tau = \{(o_t, a_t, r_t)\}_{t=1}^N,$$

T: submission attempt $\alpha \in (0, 1]$

$$R_{\text{correct}}(\tau) = r \cdot \alpha^T.$$

$$R(\tau) = R_{\text{format}}(\tau) + R_{\text{correct}}(\tau).$$

$$r^{(t)} = F_1(\hat{y}^{(t)}, y^*) \in [0, 1].$$

효율적인 탐색

정확하게 빠르게

- $|y|$: output length,
- L_{expect} : safe-zone threshold,
- L_{max} : the hard limit

$$R_{\text{format}}(\tau) = \begin{cases} 0, & |y| \leq L_{\text{expect}}, \\ -\frac{|y| - L_{\text{expect}}}{L_{\text{max}} - L_{\text{expect}}}, & L_{\text{expect}} < |y| \leq L_{\text{max}}, \\ -1, & |y| > L_{\text{max}}. \end{cases}$$

No penalty

Linearly penalty

Max penalty

Experiments

- Main Results

Table 1: Main results on seven multi-hop question answering (MHQA) benchmarks. All the results labelled with [†] are taken from (Zheng et al., 2025).

Method	Inference Environment	In Domain					Out of Domain			
		NQ	TQ	Hotpot	2Wiki	Avg	Musique	Bamb	PopQA	Avg
CoT [†]	-	32.0	48.2	27.9	27.3	33.9	7.4	21.6	15.0	14.7
CoT+RAG [†]	Local RAG	59.6	75.8	43.8	24.8	51.0	10.0	27.2	48.8	28.7
Search-o1 [†]	Web Search	55.1	69.5	42.4	37.7	51.2	19.7	53.6	43.4	38.9
7B/8B Models										
Qwen3-8B w/ Tools	Local RAG	67.0	76.4	50.8	33.0	56.8	18.4	43.2	44.0	35.2
Search-r1-base [†]	Local RAG	60.0	76.2	63.0	47.9	61.8	27.5	57.6	47.0	44.0
Search-r1-instruct [†]	Local RAG	49.6	49.2	52.5	48.8	50.0	28.3	47.2	44.5	49.5
R1-Searcher [†]	Web Search	52.3	79.1	53.1	65.8	62.6	25.6	65.6	43.4	44.9
DeepResearcher [†]	Web Search	61.9	85.0	64.3	66.6	69.5	29.3	72.8	52.7	51.6
14B Models										
Qwen2.5-14B w/ Tools	Local RAG	72.1	83.8	62.9	70.9	72.4	29.7	72.0	46.1	49.3
Qwen2.5-14B w/ Tools	Web Search	72.5	87.9	67.9	80.3	77.2	26.6	73.6	54.7	51.6
Qwen3-14B w/ Tools	Local RAG	73.1	80.9	54.9	52.5	65.4	22.7	63.2	46.7	44.2
Qwen3-14B w/ Tools	Web Search	73.7	84.2	57.9	58.5	68.6	23.2	65.6	57.7	48.8
Search-r1	Local RAG	66.9	82.6	69.8	57.0	69.1	36.9	64.8	56.3	52.7
WebSeer	Local RAG	81.9	86.7	70.9	76.0	78.9	35.0	81.6	60.6	59.1
WebSeer	Web Search	82.8	91.0	72.3	84.2	82.6	35.2	80.0	58.0	57.7

Experiments

- Main Results - challenging benchmarks(OOD)
- **Despite being trained under site-restricted search, it perform even better when deployed in the open web.**

Table 2: Evaluation on three harder benchmarks. Qwen2.5-14B and WebSeer use web search engine, while Search-r1 relies on local RAG.

Model	FanoutQA	FRAMES	SimpleQA	Avg.
Qwen2.5-14B	45.5	52.7	85.7	61.3
Search-r1-14B	12.6	29.5	36.4	26.2
WebSeer	55.4	56.1	90.0	65.3

Experiments

- **Training progressively shapes tool-use behavior from underuse to strategic deployment. @HotpotQA**
 - Pre-SFT: 3 calls → under use
 - Post-SFT : 10 calls → over use
 - Post-RL: 5 ~ 10 calls → strategic use

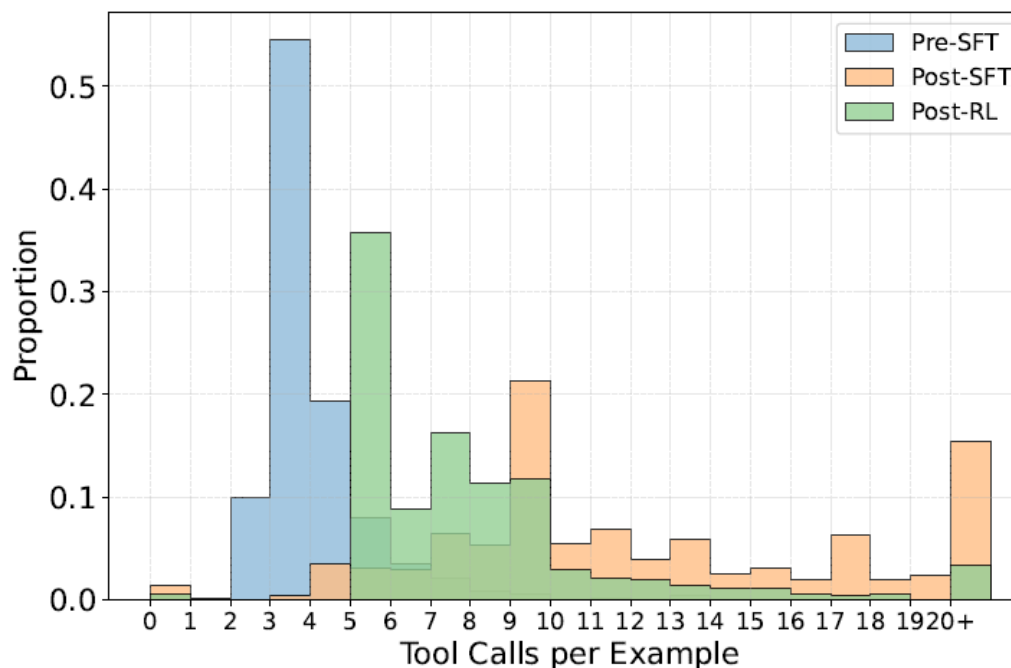


Figure 3: Tool call distributions on HotpotQA across three training stages: Pre-SFT, Post-SFT, and Post-RL.

Experiments

- Ablation Studies

Method	HotpotQA		SimpleQA	
	Acc	Tool Call	Acc	Tool Call
SFT	68.75	13.43	76.17	10.82
w/ <i>GRPO</i>	67.27	7.38	75.98	6.15
w/ <i>SRRL</i> (<i>WebSeer</i>)	70.90	7.91	78.91	8.61

Method	HotpotQA		SimpleQA	
	Acc	Tool Call	Acc	Tool Call
SRRL w/o SFT	0.00	N/A*	0.00	N/A*
SRRL w/ SFT (<i>WebSeer</i>)	70.90	7.91	78.91	8.61

*The model generates malformed output, making valid tool calls impossible.

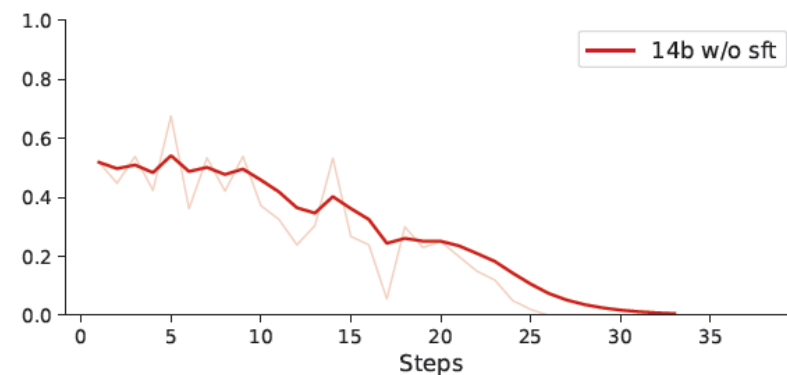


Figure 5: (Top) Limiting the model to submit only one answer results in a decrease in the final model’s performance. (Bottom) Without complex data for SFT, the 14B model collapses with decreasing rewards.

감사합니다.